

MAC0425 - EP1: Agente de Busca para o Pac-Man

Leonardo Heidi Almeida Murakami - 11260186

11 de abril de 2026

1 Estatísticas das Buscas

1.1 Cenário 1 - Encontrando a comida num unico ponto

Tabela 1: Desempenho dos Algoritmos

Algoritmo	Labirinto	Custo	Nós Expandidos
DFS	tinyMaze	10	14
	mediumMaze	130	144
	bigMaze	210	390
BFS	tinyMaze	8	15
	mediumMaze	68	269
	bigMaze	210	620
UCS	mediumMaze	68	269
	mediumDottedMaze (StayEast)	1	186
	mediumScaryMaze (StayWest)	68719479864	108
A* (Manhattan)	tinyMaze	8	14
	mediumMaze	68	221
	bigMaze	210	549
ID-DFS	tinyMaze	8	58
	mediumMaze	68	15429
	bigMaze	210	60312

Tabela 2: Desempenho do LRTA* no mediumMaze (Heurísticas Nula e Manhattan)

Trials	Heurística Nula			Heurística Manhattan		
	Custo	Expandidos	$H(s_0)$	Custo	Expandidos	$H(s_0)$
10	76	3658	16	74	2130	56
30	130	7486	30	68	5168	66
50	110	9950	39	68	6580	68
100	74	15034	59	68	9980	68
150	68	18874	68	68	13380	68

Tabela 3: Desempenho dos Algoritmos no Cenário 2 (Cantos)

Algoritmo	Labirinto	Custo	Nós Expandidos
BFS	tinyCorners	28	252
	mediumCorners	106	1966
	bigCorners	162	7949
A* (Corners)	tinyCorners	28	154
	mediumCorners	106	692
	bigCorners	162	1725

1.2 Cenário 2: Encontrando comida nos Cantos do Labirinto

2 Discussão Comparativa

O algoritmo **DFS** apresentou uma execução rápida e com baixo uso de memória nos ambientes maiores. No entanto, a principal desvantagem observada foi a incapacidade de garantir um caminho ótimo: no *mediumMaze*, encontrou uma solução de custo 130, enquanto o custo ótimo é 68.

Em contrapartida, a **BFS** explora o grafo de forma exaustiva (em largura), o que garante um plano de custo mínimo (ótimo de 68 no *mediumMaze*). Essa exploração exaustiva aumenta consideravelmente a quantidade de nós expandidos em contraste com a DFS e, por consequência, demanda muito mais memória.

A **UCS** lida adequadamente com cenários em que as etapas têm custos desiguais. No *mediumDottedMaze* (StayEast), o agente encontrou um caminho ótimo com custo 1 ao priorizar andar para a direita, em que o custo diminuía. No *mediumScaryMaze* (StayWest), em que ir para a direita aumentava o custo exponencialmente (2^x), a UCS atenuou esse grande gasto evitando essas localizações, avaliando a função de custo dinamicamente em vez de apenas passos simples.

O algoritmo **A*** parece a escolha ideal de equilíbrio entre exploração e caminho ótimo: em ambos cenários, ele garantiu o caminho ótimo alcançando os mesmos custos retornados pelas cegas minimiais (BFS, UCS), porém reduzindo muito o número de expansões do BFS quando foi guiado por boas heurísticas, poupando memória.

O algoritmo **ID-DFS** alcançou o mesmo custo da DFS com o tamanho ótimo da BFS. Para conseguir isso sem utilizar uma memória extensa de estados enfileirados, foi preciso gerar as camadas da árvore repetidas vezes a cada profundidade investigada. Esse trade-off em tempo e limite resultou no maior número geral de nós expandidos em ambientes muito extensos, como o *bigMaze* (mais de 60 mil).

Por fim, o **LRTA*** ilustrou, ao longo das tentativas repetidas (*trials*) no *mediumMaze*, como o agente calibra a estimativa. Usando a heurística nula, ele gradativamente aproximou o custo da intuição $H(s_0)$ próximo da verdade (68) pelo processo on-line, em várias iterações com custo oscilando antes de ser constante. Com a **Manhattan Heuristic**, essa curva estabiliza quase que prontamente nas explorações iniciais por começar as buscas muito perto da melhor direção verdadeira ao final para os objetivos.

3 Resposta das Questões

Questão 1: Na DFS, a ordem de avaliação dos sucessores (Norte, Sul, Leste, Oeste) define o comportamento padrão de exploração do agente. Se o algoritmo não registrasse os estados já visitados (como ocorre em uma busca em árvore pura), o agente ficaria preso em ciclos infinitos. Em vez de avançar, ele andaria de um lado para o outro entre posições adjacentes, repetindo

o mesmo caminho de ida e volta indefinidamente até estourar o limite de profundidade ou a memória do sistema.

Questão 2: A busca BFS sempre garante um caminho com o menor número de passos (custo mínimo, já que 1 passo = custo 1). Se o simulador alterar a ordem de avaliação dos sucessores, a BFS ainda encontrará uma rota otimizada. A única diferença é que ela poderá retornar um caminho alternativo, mas que possui exatamente o mesmo tamanho e custo total, garantindo a solução ótima.

Questão 3: No cenário *StayEast*, ir para a esquerda tem um custo alto, enquanto caminhar para a direita diminui o custo. Por isso, a UCS (Busca de Custo Uniforme) prefere rotas que explorem mais o lado direito, minimizando o custo total acumulado. Por outro lado, no cenário *StayWest*, andar para a direita gera uma penalidade exponencial (2^x). Nesse caso, a UCS muda a estratégia e passa a contornar o mapa pela esquerda, buscando evitar a todo custo os passos que geram multas tão altas.

Questão 4: A principal vantagem do A^* em relação à UCS é que ele não considera apenas o custo real do caminho já percorrido ($g(s)$), mas também usa uma estimativa (heurística) de quão perto o agente está do objetivo ($h(s)$). Para que o A^* sempre encontre o melhor caminho, essa heurística precisa ser consistente. Isso quer dizer que a distância estimada de um ponto até o de destino nunca deve criar atalhos irreais maiores que o custo real de locomoção.

Questão 5: No algoritmo LRTA*, o agente atualiza seu conhecimento sobre o labirinto de forma contínua enquanto se movimenta, ajustando suas estimativas com base na experiência (aprendizado on-line). Quando usamos a heurística de Manhattan, o agente já entra em campo com uma boa noção espacial. Isso significa que ele comete bem menos erros e estabiliza no caminho ótimo muito mais rápido. É um comportamento bem diferente do que acontece com a heurística nula, na qual o agente dá "pulos às cegas" nas primeiras tentativas até começar a memorizar o melhor trajeto.

4 Melhorias e Dificuldades

A principal dificuldade no desenvolvimento foi no problema dos cantos, em que foi preciso encontrar uma representação de estado que fosse leve e eficiente. Em vez de passar instâncias completas e pesadas da classe *GameState* para cada nó da busca, optei por utilizar tuplas contendo apenas as informações essenciais (como a posição do agente e os cantos restantes). Essa escolha evita um alto consumo de memória e garante um ganho substancial de desempenho.